QECTOR Decoder Workbench

MCP INTEGRATION GUIDE

Application version 1.0.1
Decoder backend qector-decoder-v3 1.0.0 (bundled wheel, activated offline on first launch; minimum supported 1.0.0)
85-tool MCP server, 17 decoders, 10 code families

Guillaume Lessard / iD01t Productions. https://www.qector.store
Generated 2026-08-19

Contents

1. Overview

QECTOR Decoder Workbench embeds an MCP (Model Context Protocol) server that exposes 85 tools for building codes, decoding, benchmarking, streaming, documentation export and system introspection. Any MCP-capable client, including AI assistants and custom agents, can drive the workbench headlessly.

Server name	qector-workbench
Transport	newline-delimited JSON-RPC 2.0 over stdin/stdout (stdio)
Protocol version	2024-11-05
Tool count	85
Launch flag	--mcp

2. Launching the server

```
Windows portable : QectorWorkbench-Portable.exe --mcp
Linux (.deb) : qector-workbench --mcp
macOS app : /Applications/QectorWorkbench.app/Contents/MacOS/QectorWorkbench --mcp
From source : python mcp_server.py
```

3. Client configuration

Most MCP clients accept a small JSON entry describing the command and arguments. The examples below register QECTOR as a stdio server. Use the absolute path to the executable on the target machine.

Windows client configuration

```
{
 "mcpServers": {
 "qector": {
 "command": "C:\\Apps\\QectorWorkbench-Portable.exe",
 "args": ["--mcp"]
 }
 }
}
```

Linux or macOS client configuration

```
{
 "mcpServers": {
 "qector": {
 "command": "qector-workbench",
 "args": ["--mcp"]
 }
 }
}
```

4. The handshake

A session opens with an initialize request, an initialized notification, and then tool discovery. One JSON object per line.

```
{"jsonrpc":"2.0","id":1,"method":"initialize","params":{"protocolVersion":"2024-11-05","capabiliti
{"jsonrpc":"2.0","method":"notifications/initialized"}
{"jsonrpc":"2.0","id":2,"method":"tools/list"}
```

5. Calling a tool

Invoke a tool with tools/call, passing its exact name and an arguments object. A minimal build-and-decode flow:

```
{"jsonrpc":"2.0","id":3,"method":"tools/call","params":{"name":"build_code","arguments":{"family":
{"jsonrpc":"2.0","id":4,"method":"tools/call","params":{"name":"run_decode","arguments":{"decoder"
```

Note. Tool and argument names must match exactly. Use only the decoder kinds and code families listed in this documentation set; any other value returns a handled error.

6. Interpreting results and errors

syndrome_valid	True only when the correction reproduces the syndrome (H times c equals s mod 2). Success requires this.
logical_failure	True, false or null. Null when a code exposes no logicals matrix (the qLDPC families).
logical_error_rate	A simulation estimate for the seed, workload and hardware; not a universal benchmark.
isError true	A tool-level error. Relay the message text; do not treat it as success.
array summaries	Large arrays may be summarised (shape, dtype, preview). Do not infer omitted values.

7. Rules for agents

These rules keep an automated caller correct and prevent hallucination. They are also encoded, per tool, in QECTOR_LLM_Manual.json.

- Call only tools whose exact name is listed in this manual. Never invent a tool name, a parameter name, a decoder kind, or a code family. Unknown names return an error.
- Use only the decoder kinds in the decoders list and the code families in the families list. Passing any other value returns a handled error, not a result.
- Report tool errors exactly as returned. When a tool result carries isError true, relay the message text and do not fabricate or guess a successful outcome.

- Do not claim a decode succeeded unless the returned object has syndrome_valid true. Every accepted correction satisfies the parity-check equation H times c equals s modulo 2; do not assert any property that is not present in the returned object.
- Treat logical error rate, throughput and latency as simulation outputs that depend on the seed, the hardware, the driver and the workload. Never present them as fixed or universal benchmarks. State that they must be regenerated for the target setting.
- The qLDPC families bicycle and bivariate_bicycle are not graphlike. Decode them with bp_osd, blossom, sparse_blossom, hybrid, predecoded, auto or auto_router. The union_find, fast_union_find and lookup_table decoders do not apply to them.
- Version numbers are resolved live at runtime. Do not state a version without first reading it from the version_info or get_system_info tool.
- The decoder wheel is bundled inside the application. On first launch the app extracts and activates the bundled qector-decoder-v3 wheel into an ABI-scoped managed site, entirely offline. Any outdated managed decoder from an older release is purged automatically before activation. Do not claim the app downloads anything at runtime.
- OpenCL is reported unavailable by the published qector-decoder-v3 wheel because that wheel is built without its OpenCL feature, not because the machine lacks a GPU. No environment variable can turn it on. Do not tell a user to fix drivers or set a flag to enable it; CUDA and CPU are unaffected.
- When a large array in a result is summarised or truncated, do not extrapolate or invent the omitted values. Work only from what the result actually contains.
- The destructive tools (for example those that clear results, delete a resource or reset configuration) require confirm set to true. Do not call them unless the user explicitly asked for that action.
- If a tool call fails or times out, say so plainly. Do not present a placeholder or an assumed value as though it came from the server.

8. Tool reference

All 85 tools, alphabetical. Each tool's full parameter schema and an example call are in QECTOR_LLM_Manual.json.

ambiguity_cluster_decode	Decode using AmbiguityClusterDecoder for high noise or non-graphlike codes
analyze_code_family	Analyze a code family with an example code instance
analyze_error_patterns	Analyze error patterns: weight distribution, cluster size, correlated errors
analyze_logicals	Expose logical operator matrix, logical weight distribution, and code distance estimation
batch_decode	Batch-decode sampled syndromes on cpu/cuda/opencl via backend.run_batch_decode
batch_decode_gpu	Batch-decode on an explicit compute backend (cpu/cuda/opencl) with honest availability reporting, unavailable GPU backends return status='unavailable' with a reason, never fake results
belief_match_decode	Convenience seeded decode pinned to the belief_matching kind (BP posteriors + exact MWPM with faithfulness fallback)
benchmark_decoder	Benchmark a decoder on a code family via backend.run_benchmark (latency percentiles, throughput, logical error rate)
build_code_from_matrix	Build a code from a user-provided parity check matrix
build_dem	Build a Detector Error Model (DEM) from a code and noise model
check_updates	Report whether the installed decoder backend matches the bundled release baseline (offline, no update service)
clear_decoder_cache	Clear the backend's native decoder cache
clear_results	Clear all stored benchmark results
colour_code_decode	Decode color code using BP-OSD over undecomposed detector error model
compare_all_decoders	Run all compatible decoders on the same code and return comparison table
compare_benchmarks	Compare stored benchmark results side by side (throughput, p99 latency, logical error rate)
compat_report	Report ecosystem-integration availability (stim/sinter/pymatching/qiskit/ldpc) and research components
compatibility_matrix	Return the full 16x10 decoder/code compatibility matrix
compatible_decoders	Live probe: which decoder kinds construct and produce a syndrome-verified correction on this code
compliance_attestation	Zero-egress / offline compliance attestation for infosec review: AST scan for network and telemetry imports, runtime EgressGuard state, offline license tier, local-only data residency, and optional Entra ID readiness. No network calls.
decode_dem	Decode using a Detector Error Model (DEM-native decoding)
decode_hyperedge	Hyperedge / qLDPC decoding via bp_osd or other LDPC-capable decoders
decode_mmap	Out-of-core batch decoding via memory-mapped arrays
decode_single	Run one seeded decode and report correction weight, syndrome validity and logical failure

decode_syndrome	Decode an explicit 0/1 syndrome (length n_checks) with a chosen decoder; syndrome_valid is the GF(2) re-check, logical_failure is null (no reference error exists, so it is unknowable)
decode_syndrome_blossom	Convenience tool: exact Blossom (MWPM) syndrome decode
decode_syndrome_cascade	Convenience tool: Hybrid cascading syndrome decode (UF pre-filter escalating to Blossom)
decode_with_options	Seeded decode with validated per-decoder construction options (bp_osd bp_method/osd_order, hybrid_cascade escalation, GNN architecture); reports options_applied honestly
decoder_benchmark_suite	Run standard benchmark (rotated_surface d=5, p=0.05) across all decoders
delete_resource	Delete a resource by ID
diagnostic_decode	Rich single decode via the backend's native decode_with_diagnostics (matched weight, backend used, internal fallback, timing, logicals)
doctor_diagnostics	Run system health and environment diagnostic checks via qd.doctor
estimate_threshold	Estimate the error threshold using binary search on error rate
export_benchmark	Export a stored benchmark result (by result_id) to the export directory
export_figure	Export a publication-ready figure of the Tanner graph
export_session	Export the current session (code + decode + benchmark + diagnostics) as a ZIP archive
finite_size_scaling	Perform finite-size scaling analysis (LER vs distance at fixed p)
flush_usage	Flush usage metrics to Stripe metered billing API
generate_documentation	Generate code documentation files
generate_parity_check	Generate a parity check matrix for a code family
generate_reproducibility_package	Generate a complete reproducibility package (ZIP)
get_backend_health	7-tier backend health status from AutoDecoder diagnostics
get_code_properties	Get properties of a code family
get_config	Get current server configuration
get_decoder_info	Get information about a decoder
get_entra_posture	Return the Microsoft Entra ID posture (enabled, unconfigured, or authenticated).
get_hardware_info	Get hardware/backend availability
get_identity_info	Return identity info if signed into Entra ID, else None.
get_license_info	Get license info from the decoder (tier, key_status, expiry)
get_resource	Get a specific resource by ID
get_resources	List all resources
get_results	Get stored benchmark results (most recent first-in order)
get_server_env	Get effective QECTOR environment variables (tuning vars)
get_statistics	Get server statistics
get_system_info	Get system information
gnn_belief_match_decode	Convenience seeded decode pinned to the gnn_belief_matching kind with optional GNN architecture overrides (gnn_hidden_size, gnn_n_layers)

hybrid_cascade_stats	Batch-decode through the hybrid_cascade decoder and expose its live cascade statistics (prefilter_hits, escalations, hit rate, throughput, syndrome-match rate, logical error rate)
import_stim	Import a Stim circuit from file and convert to DEM
import_syndrome	Load external syndrome data (CSV, JSON, or .npy) and decode it
list_clients	List registered clients
list_code_families	List available code families
list_codes	List workbench code families plus the backend's native code catalogue
list_decoders	List available decoders
list_tools	List all available MCP tools
mcp_health	Server health check: uptime, memory, decoder status, tool count
mcp_status	Get MCP server status
native_recommend	Backend-native decoder recommendation (qector_decoder_v3.recommend) with the mapped workbench decoder_kind
native_streaming	Native hardware-accelerated sliding-window streaming decode (qector_decoder_v3.sliding_window_decode) with per-round validity + telemetry
neural_predecoder_train	Train the NeuralPredecoder research/lab MLP on seeded (syndrome, error) pairs and evaluate on a disjoint held-out stream (exact-match, bit accuracy, syndrome validity, LER)
parallel_batch_decode	Parallel batch decode using multiple processes via backend.run_parallel_batch_decode
probe_decoders	Probe which decoders produce a valid (syndrome-verified) correction for a code, a self-test across every wired decoder
recommend_decoder	Recommend a decoder for a code/priority using detected hardware
register_client	Register a client
reset_config	Reset configuration to defaults
resilient_decode	Single decode with automatic multi-decoder fallback and a full attempt trace (autodebug.resilient_single_decode)
run_benchmark	Run a benchmark and store the result under a generated result_id
run_ler_benchmark	Run LER benchmark with Wilson confidence intervals
self_diagnostics	Run a full environment/decoder/hardware self-diagnostics report (autodebug.run_self_diagnostics)
set_config	Merge key/value pairs into the server configuration
set_license_key_file	Set license key file path for offline verification
sparse_blossom_radix_neighbors	Discover k-nearest candidate edges via SparseBlossom RadixHeap
stream_decode	Run a sliding-window streaming decode session via backend.run_streaming_session
two_stage_decode	Decode using TwoStageDecoder (decoupled X/Z sector decoders)
verify_license_token	Verify an Ed25519 signed license token string
version_info	App + decoder-backend version report (workbench baseline + installed backend, resolved locally, no network)